Computer Architecture and System Software

Lecture 07: Assembly Language

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Assignment 3 released
  - 3 assembly coding questions
  - Due Mar 13th at 11:59pm
  - After today's lecture, should be able to complete all questions

Flags and Stack Instructions

Flag Instructions

- CLC – Clear carry flag
- STC – Set carry flag
- CLD – Clear direction flag
- STD – Set direction flag
- CLI – Clear interrupt flag
- STI – Set Interrupt flag
- CMC - Complement Carry flag
- LAHF – Load AH from 8 low bits of flags register

The Stack

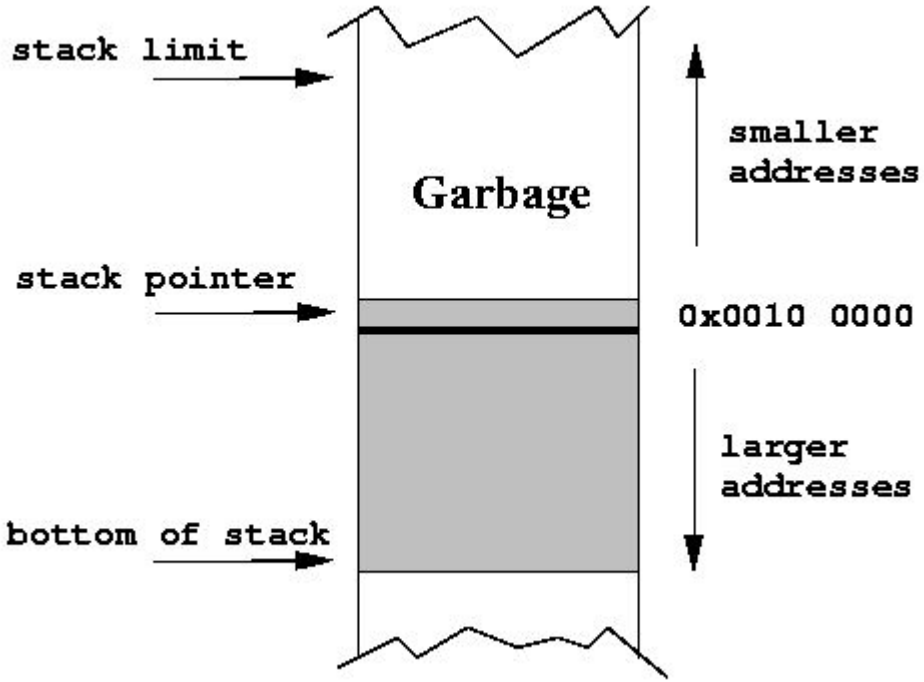
- Operates on first in - last out principle
- Portion of memory used to store local variables
  - When you define an int, for example
- Can use this memory space to temporarily store results, if we run out of room in our registers
- Registers BP/SS (base pointer and stack segment) always point to the top/bottom memory address of the stack

Stack Instructions

- PUSH operand
  - Operand could be reg, memory, or immediate
- POP operand
  - Operand could be reg, memory
- PUSHA – push all general purpose registers onto the stack
  - AX, CX, DX, BX, SP, BP, SI, DI
- POPA – get all general purpose registers from the stack
- PUSHF – store flags register in the stack
- POPF – get flags register from the stack

The Stack

- How does the stack evolve?
  - Stack grows towards smaller addresses
  - Stack size is static and predetermined
  - Stack Base Pointer register (BP) points at the bottom of the stack
  - Stack Pointer (SP) points to top of stack

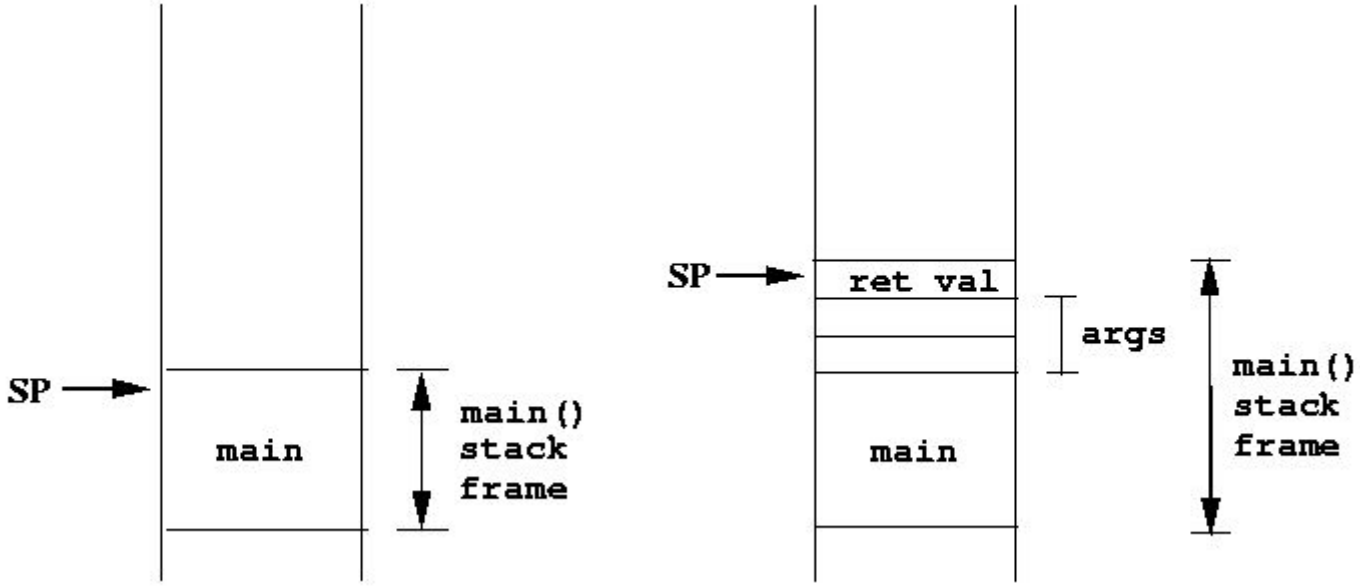


The Stack

- If stack grows beyond stack limit we have *stack overflow*
- Upon a stack overflow in modern computers, the OS steps in and kills program (protecting memory spaces of other processes)
- In DOSBOX, process will keep running after stack overflow.
- Let's look at a high level view of the stack during execution of a C program

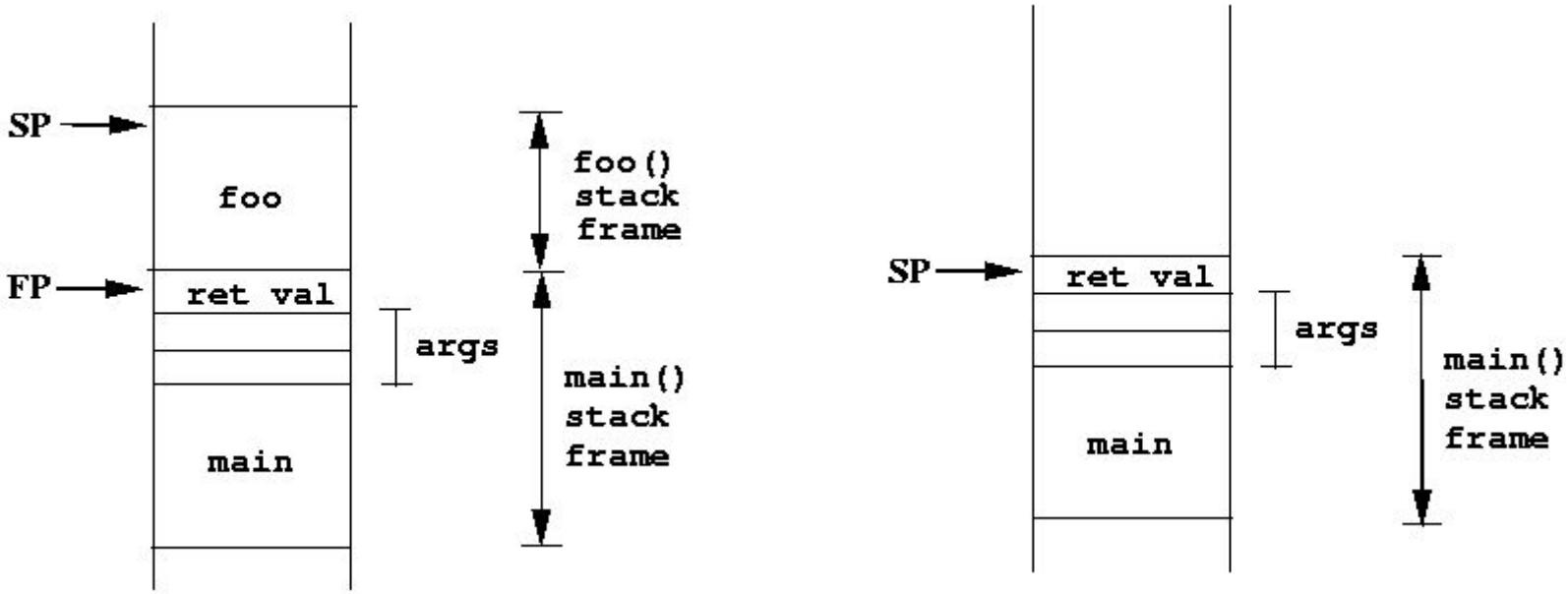
The Stack

- Main() program assigned its own stack frame
  - Size depends on local variables
- Main() will call another function, and will pass two arguments. These arguments are pushed to the stack
- Space is also reserved for return value of function



The Stack

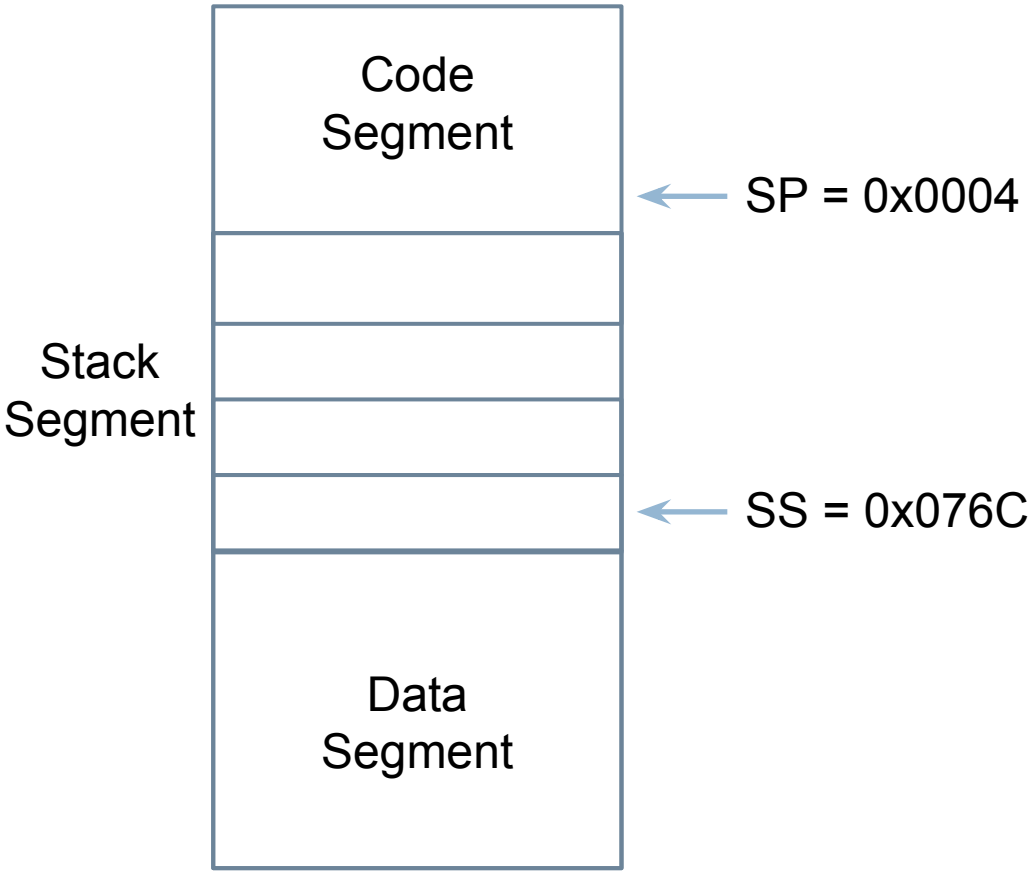
- Called function foo() gets its own stack frame for any local variables
- Upon completion, program returns to instruction after program call and the stack shrinks



The Stack

- Stack can get out of hand if you are using too many recursive functions
- Can access any point in stack using BP/SS registers
  - Mov bp, 0
    mov ax, [bp]
    mov ax, SS:[0]
- Stack does not handle dynamic memory allocation, this is handled by the heap (not covered in this course)
  - eg. malloc()

The Stack



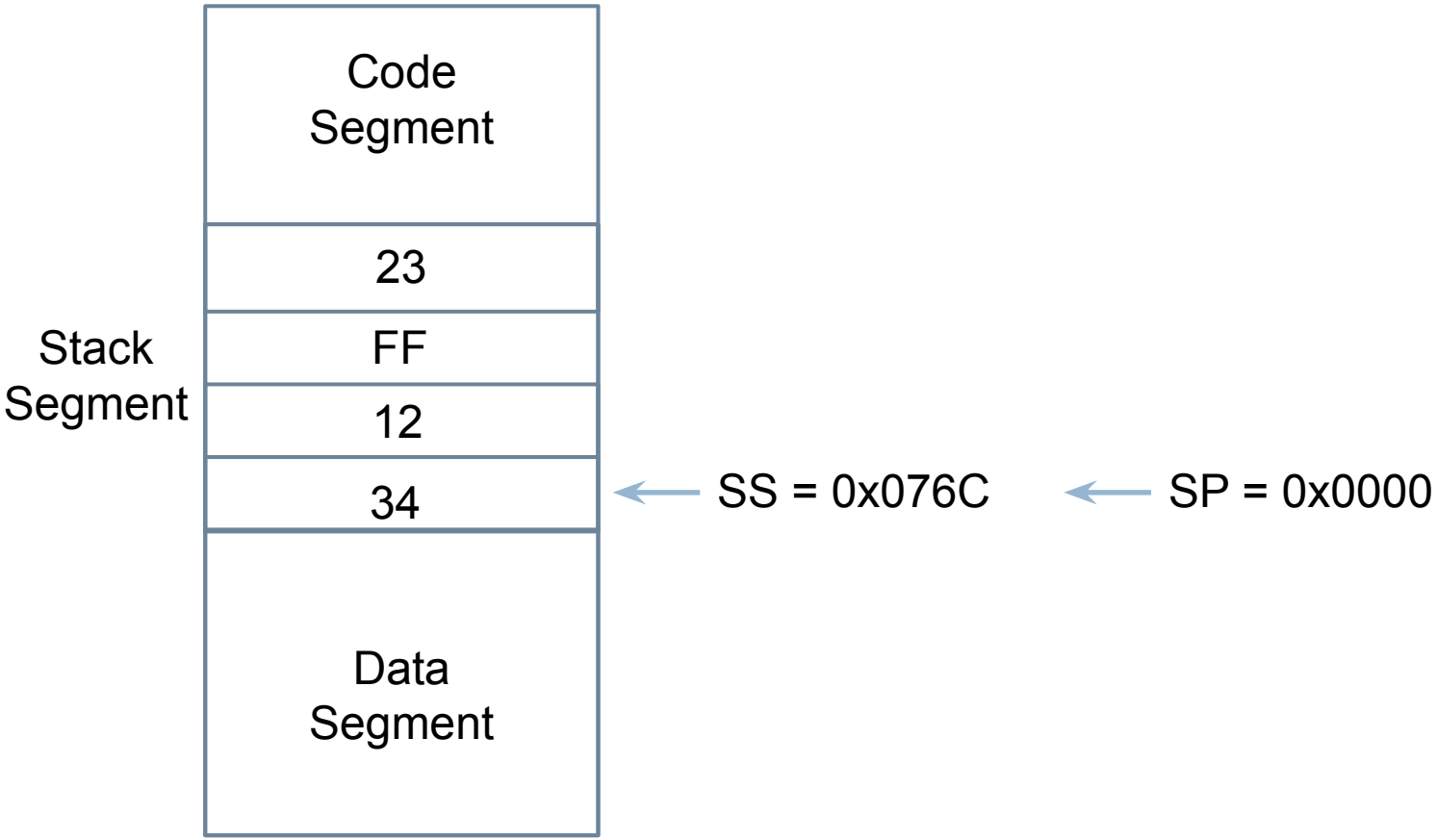
- Let's look at a low level view of the stack
- Before execution, SS points to base of stack (smallest address)
- SP points to top of stack (largest address)
- Note SP holds an effective address, while SS holds a physical address.

The Stack

- In the previous example, the stack segment was 4 bytes large.
- When the stack is empty, SP points to SS with an offset of [stack size].
- Pushing data onto the stack decrements SP, since the stack grows towards smaller addresses.
- Code and Data segments are of arbitrary size.

The Stack

- Push dx ; dx = 23FFh
- Push cx ; cx = 1234h



The Stack

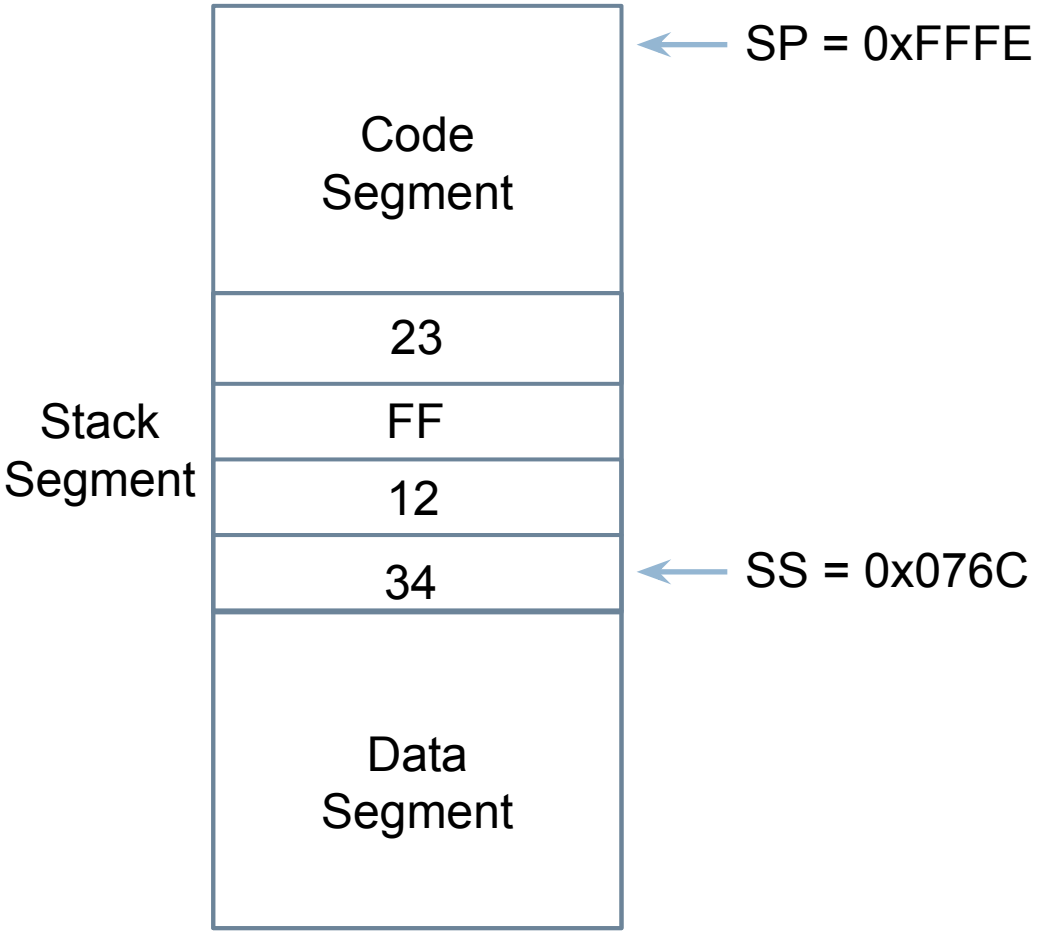
- Since we have 16 data pins, we must push 16-bits at a time. This initializes two bytes of the stack and SP is decremented twice for each push.

- Little endian architecture Low order bits stored in smaller memory addresses.

- What happens if we keep pushing dx?
  - SP decremented from 0x0000 to 0xFFFE (stack overflow)
  - Register pushed to SS:FFFE

The Stack

- Push dx ; dx = 23FFh



Dx pushed somewhere in code segment. Effects may not be noticeable until much later!

More on Segments and addressing

- Main memory is a **linear array of bytes**
- Recall that the x86 architecture has a 20-bit address bus
- 16 bits are used as an offset from base address
- Base address is a 16-bit number stored in (ES,SS,DS,CS) segment registers
- Where does the extra 4-bits come from?

More on Segments and addressing

- Segment addresses are shifted 4 bits to the left before adding the offset
- Thus, if SS = 0x076C and SP = 0x0004

(0000 0111 0110 1100 << 4)
+ 0000 0000 0000 0100

- = 0000 0111 0110 1100 0000
  + 0100
- = 0000 0111 0110 1100 0100
- = 20-bit address given by SS:0004

More on Segments and addressing

- It is possible for two [segment:offset] pairs to map to the same address
- Example: SS = 0x076A, CS = 0x076B
  - SS:0020 and CS:0010 map to same address
- Little protection for this in 8086 architecture aside from good programming practices
- Programmer chooses size and base address of segments in assembly template

Procedures

Procedures


```
push ax              ;push contents of register ax to stack
push dx              ;push contents of register dx to stack
call myproc          ;call (procedure name)
…                    ;instruction executed after procedure

; procedure
myproc proc              ;(procedure name) proc
    mov ah, 02h
    mov dl, 'S'          ;code for displaying 'S'
    int 21 h
    ret              ;return to instruction after call
myproc endp              ;(procedure name) endp
```

Procedures (Background)

- The Pentium stack is defined to grow towards smaller addresses
- Stack Pointer (SP) points to the top of the stack
- Stack Segment (SS) points to the start of the stack segment in memory
- Call
  - Return address (IP) is pushed to stack
  - Jump to effective address given by operand
  - Return address is the address of the instruction following the call

Procedures (Background)

- Return
  - Pop return address from stack
  - Jump to that address

- Note: Procedures are defined after they are called, (after the final int 21h command) but they must be defined before the end of the code segment.

Parameter Passing

- There is little/no support for passing parameters to procedures
- When it comes to implementation
  - Convention
  - Its up to the programmer to follow conventions

Parameter Passing

- Passing parameters means getting data into a place set aside for the parameters
- Both the calling program and the procedure need to know where the parameters are
  - Calling program places them
    - Possibly uses values returned by the procedure
  - Procedure uses the parameters

Parameter Passing

- Simplest mechanism: use registers
  - Calling program puts the parameters into specific registers
  - Procedure uses them
    - Int 21-DOS function dispatcher does this

Parameter Passing

- Intel architecture suffers from not having enough registers
  - Very soon one runs out of registers to use
- Parameter passing by registers is not used as much with this method
  - Used on more modern architectures
- Solution: pass parameters on stack

Parameter Passing

- One last problem: What happens if procedure needs to use registers?
- Solution: store registers on stack before using them
- Two ways of implementing this:
  - Callee saved: a procedure clears out some registers for its own use
  - Caller saved: The calling program saves the registers and local variables that it does not want procedure to overwrite

Parameter Passing

- Either way you may need to use pusha, pushf, popa, popf
- Note: MASM generates an error message if using the PUSHA or POPA
- By default the MASM generates code for the 8086 processor
  - These instructions are implemented only for the 80186, 80286, 80386 processors.
- To solve problem use a .186, .286, or .386 directive in the first line of your code
- See "Error A2105 with PUSHA and POPA instructions“

IP register and RET

- Calling a procedure pushes the IP register to the stack
- This is later popped from the stack at ret, so that the assembly program can continue executing instructions where it left off before the call
- Therefore, procedures should not push to stack without popping – otherwise the wrong address will be popped to the IP register

An infinite loop


```
mov dx, 0      ;dx = 0
call myproc  ;push IP register to stack

myproc proc
push dx        ;push 0 to stack

ret        ;pop 0 to IP register
myproc endp      ;procedure ends. Next instruction to
be executed will be at address CS:00
            (Effective address 0 of code segment)
```

Recursion

Recursion

- Procedures may call themselves
- Process is similar to recursion in C (Java)
- Procedure call pushes IP register and sets IP to beginning of procedure
- Can be tricky to implement because limited space in registers

- Example: recurs.asm

Arrays

Arrays

- Arrays in assembly are 1D, and are defined in the following ways:

```
myArray db 8 dup(?)          ;8 element uninitialized array
myArray db 8 dup(1)          ;8 element array
                             ;all elements initialized to 1
myArray db 0,1,2,3,4,5,6,7       ;8 element array
                                 ;[0 1 2 3 4 5 6 7]
myArray dw 4245h, FFFFh, 0017h,C642h     ;8 element array
                             ;[45h 42h FFh FFh 17h 00h 42h C6h]
```

Arrays

- As with strings, the effective address of the array is the memory address of the first element

- Load effective address instruction (lea) can be used in the same way
  - Lea dx, myArray ;loads effective address of ;array in dx

- Remember indexing into memory must be done using registers si, di, or bx.

Arrays

- Using register indirect addressing, we can easily access elements of the array
  - mov al, myArray[4] ;Move array element 4
    ; to register al
- As always, be careful about operand sizes and memory to memory operations, which are forbidden
  - mov myArray[2], myArray[5] ;mem to mem forbidden
- The offset [num] is the offset in **bytes**, regardless of whether the array was defined using db/dw/dq

Arrays

- See array.asm

Reading large numbers

- Recall that we were able to read, display and perform arithmetic with one digit numbers
- We concluded that multi-digit numbers would have to be decoded one digit at a time
  - There is no ASCII character for '10', so we process '1' and '0' separately
- Let's look at the general procedure for the reading of a multi digit number

Reading numbers

- Steps:
  1. Read string of numbers into buffered input
  2. Point to last character in string
  3. Convert using ASCII, multiply by a power of 10
  4. Store result, check if done
  5. Otherwise, point to next character and repeat steps 3 and 4

Printing numbers

- Steps:

  1. Integer division of a large number by 10 gives 'ones' place as remainder

     (13/10 = 1 + 3R)

  2. Convert remainder to ASCII and push to stack

  3. If quotient is nonzero, repeat steps 2-3 on the quotient.

  4. Pop chars off the stack one at a time and print (one pop for every division)

Reading/printing numbers

- Let's step through the procedure print_int slowly on the board with the "13" example

- This will be a handy procedure for you in future assignments/labs.

Extending to 16-bit functionality

Extending 8-bit program to 16-bit

- Example:
  - **8-bit** multiplication (*mul bl*)
    - ax = al×bl
  - **16-bit** multiplication (*mul bx*)
    - [dx ax] = ax × bx
  - **8-bit** integer division (*div bl*)
    - al = ax÷bl, ah = remainder
    - Note: Program will hang if ax ÷ bl is larger than 8-bits
  - **16-bit** integer division (*div bx*)
    - ax = [dx ax] ÷ bx, dx = remainder

Extending 8-bit program to 16-bit

- In the last example, multiplication required the use of 3 registers.
- Space very limited once data sizes become large
- Use the stack!

```
push ax   ;Save ax
push dx   ;Save dx
mul bx    ;Multiply – overwrites ax, dx
…         ;Other instructions
pop dx    ;Restore dx
pop ax    ;Restore ax
```

Extending 8-bit programs to 16-bit

- Alternatively, reserve space in data segment

```
;(Data Segment)
temp dw 1 dup(?)  ;Reserve a word (2 bytes)

;(Code Segment)
mov temp, dx   ;save dx in temp variable
mul bx         ;multiplication overwrites ax,dx
…
mov dx, temp   ;restore dx from temp
```

Floats in Assembly

Floats in assembly

- In this course we have only introduced integer and bit-wise instructions.
- We will not implement any floating point arithmetic in assembly in this course.
- Instead, we will briefly overview the hardware components of the x86 architecture responsible for floating point operations.

Floats in assembly

- Original x86 processors had a separate coprocessor for FP arithmetic
  - Called x87 chips
- Floating point registers were 80-bits wide (long double size)
- 8 FP registers are organized into a stack called ST0-ST7.

ST0
ST1
ST2
ST3
ST4
ST5
ST6
ST7

Floats in assembly

- General purpose registers can not be used for floating point operations
- Special floating point instructions push and pop operands to and from the x87 register stack. Most instructions take first couple stack elements as input by default.
- Our debugger cannot view x87 registers directly. Must pop x87 registers to data/stack segment and view by dumping memory.

Floating Point Instruction Set

- Full instruction set if interested:
  www.idc-online.com/technical_references/pdfs/electronic_engineering/Data_Types_and_Instruction_Set_of_8087.pdf
- Basic:
  - FADD op - ST[0] = ST[0] + op (op can be memory or x87 reg)
  - FADDP op - same as above, but pop the x87 stack after

  - FSUB op - ST[0] = ST[0] - op (op can be memory or x87 reg)
  - FSUBP op - same as above, but pop the x87 stack after

  - FLD op – load ST[0] with op (32- or 64-bit operand from mem)
  - FST op – move data from ST[0] to op (32- or 64-bit operand)
  - FSTP op – same as above but pop the x87 stack after